

# EventBus 事件系统设计说明

## 简介

EventBus 是一个设计用于 Unity 项目的事件派发系统，支持在游戏中进行解耦的消息传递。系统提供全局事件总线和本地事件总线两种作用域，并通过标签将事件按语义聚合。事件总线结合表格定义和运行时注册，可以同时服务策划配置和程序逻辑。

## 设计目标

- **解耦通信**：系统之间通过发布/订阅事件通信，无需直接引用彼此。
- **作用域控制**：支持全局事件总线（共享信息）和本地事件总线（子树内部协作）。
- **数据驱动**：策划可以通过表格声明事件和标签，程序在初始化时加载并校验。
- **语义聚合**：通过给事件绑定标签，实现按语义订阅事件流，例如监听所有“GameOver”相关的事件。
- **动态扩展**：运行时可以注册自定义事件并绑定标签；订阅可随时注册或取消。

## 核心概念

### 全局事件总线和本地事件总线

- **Global Bus**：用于广播游戏生命周期、流程控制等公共事件。任何系统都可以获取它的引用并订阅或发布事件。
- **Local Bus**：挂在某个 GameObject（如玩家、Boss 等）上，仅其子对象可以访问。用于内部系统协作。需要对外公开的事件可通过桥接发布到全局总线并打上合适的标签。

### 事件 (Event)

事件定义了某个发生的事实，由一个字符串 `eventName` 唯一标识。事件可以通过表格声明，也可以在运行时注册。事件可以绑定多个标签用于聚合订阅。

### 标签 (Tag)

标签代表一类高层语义，例如 `LevelEvent`、`GameOver`、`Analytics` 等。每个标签在标签表中注册后方可使用。事件可以绑定任意数量的标签。订阅者可以订阅具体事件，也可以订阅标签以获取所有绑定该标签的事件。

### 信封 (Envelope) 与 Payload

派发事件时，会生成一个信封对象，其中包含：

- `eventName`：事件名。
- `tags`：绑定的标签集合。
- `busOwner`：总线所在对象的 Transform。
- `sender`：触发事件的对象的 Transform（可选）。
- `time`：时间戳。
- `payload`：携带的数据对象。可以为强类型实例或弱类型键值对。如果派发时未提供 payload，则使用默认占位对象。

订阅事件时，事件订阅者仅收到 `payload`；订阅标签时，订阅者收到整个 `Envelope`，可通过 `eventName` 区分具体事件。

## 使用方法

1. **初始化运行时数据**：在游戏启动时调用 `DataEntityRuntimeLoader.Initialize()` 等方法初始化数据表系统。
2. **配置组件**：在需要的 `GameObject` 上添加 `EventBus` 组件，并在 Inspector 中设置以下字段：
3. `mode`：Global 或 Local。
4. `eventTableTemplateName`：事件表模板名称。
5. `tagTableTemplateName`：标签表模板名称。
6. `initTagFilters`：初始化筛选条件。使用 '|' 表示 AND，不同列表元素之间为 OR。
7. `enableScopeCheckForLocal`：是否开启本地订阅作用域检查。
8. `allowTriggerTagAtRuntime`：是否允许运行时调用测试接口。
9. `logVerbose`：是否输出详细日志。
10. **显式初始化**：在启动代码中调用 `Init(IDataTableRuntime dbRuntime)`。该方法会从表格加载标签和事件，应用过滤器，并填充内部结构。
11. **订阅事件或标签**：
12. `SubscribeEvent(owner, eventName, callback)`：订阅具体事件，回调签名为 `Action<object>`，接收事件 payload。
13. `SubscribeTag(owner, tagName, callback)`：订阅标签，回调签名为 `Action<Envelope>`，接收信封。返回值 `SubscriptionToken` 可用于取消订阅。
14. **发布事件**：
15. `PublishEvent(sender, eventName, payload)`：派发事件。`sender` 可为空；`payload` 可为空。
16. 如果事件名称未声明，会打警告但仍然派发。
17. **注册自定义事件**：
18. `RegisterCustomEvent(eventName, tagString)`：在运行时注册新的事件并绑定标签。`tagString` 使用 '|' 或 ',' 分隔。无效标签会被忽略并产生警告。
19. **取消订阅**：
20. `Unsubscribe(token)`：取消指定订阅。
21. `UnsubscribeAll(owner)`：取消某个订阅者的所有订阅。
22. **测试接口**（仅用于调试）：
23. `TriggerTagForTest(sender, tagName, reasonEventName, payload)`：手动触发某个标签的订阅者，绕过正常事件语义。生产环境不应使用。

## 表格结构

### 事件表 (Event)

- **name**：事件名称（唯一）。
- **tags**：标签列表，用 '|' 或 ',' 分隔（可选）。
- 其他列可根据游戏需求扩展。

### 标签表 (EventTag)

- **name**：标签名称（唯一）。

在初始化时，EventBus 会加载标签表，将所有 `name` 字段加入标签白名单；再加载事件表，过滤事件，并声明匹配的事件。事件中绑定的标签必须存在于标签表，否则会被忽略并输出警告。

## 过滤规则

`initTagFilters` 用于控制某个 EventBus 初始化时声明哪些表定义事件：- 列表为空：声明全部事件。- 列表有元素：只声明满足至少一个过滤条件的事件。- 一个过滤条件由 `'|'` 分隔的标签组成，表示事件必须同时拥有这些标签（AND）。- 不同过滤条件之间为 OR。例如 `["LevelEvent", "Global|Lifecycle"]` 表示声明所有包含 `LevelEvent` 标签的事件，或同时包含 `Global` 和 `Lifecycle` 的事件。

## 性能与注意事项

- 订阅取消采用令牌管理和反向索引，性能与订阅者数量成正比。
- 本地总线启用作用域校验后，会在订阅时验证订阅者是否位于总线对象的子树，外部系统无法直接订阅本地事件。
- 默认为运行时事件提供一个静态的 `NoDataPayload` 单例，避免频繁分配。
- 订阅和取消订阅为低频操作，不会对帧率造成明显影响；发布事件的派发在字典和列表中查找，效率很高。

## 示例

```
// 初始化
EventBus playerBus = playerObject.GetComponent<EventBus>();
playerBus.mode = EventBus.BusMode.Local;
playerBus.eventTableTemplateName = "Event";
playerBus.tagTableTemplateName = "EventTag";
playerBus.initTagFilters = new List<string> { "PlayerEvent" };
playerBus.Init(dbRuntime);

// 注册事件订阅
SubscriptionToken hitToken = playerBus.SubscribeEvent(this, "Player.Hit",
payload =>
{
    // payload 是事件随附的数据
    Debug.Log("玩家被击中！伤害：" + payload);
});

// 注册标签订阅
SubscriptionToken gameOverToken = globalBus.SubscribeTag(this, "GameOver",
envelope =>
{
    Debug.Log($"游戏结束！事件：{envelope.eventName}");
});

// 发布事件
playerBus.PublishEvent(transform, "Player.Hit", 10);

// 运行时注册自定义事件
playerBus.RegisterCustomEvent("Player.CustomEvent", "PlayerEvent|Debug");

// 取消订阅
```

```
playerBus.unsubscribe(hitToken);  
globalBus.unsubscribeAll(this);
```

---